2013. Detect Squares

dium | Category: Math & Geometry | LeetCode: https://leetcode.com/problems/de

Problem

- Add(point) stores points (duplicates allowed). count(query) returns how many ways to choose three stored points that form an axis-aligned square of positive area with the query point.

Intuition

- Fix the diagonal: any stored point (x, y) with |x - qx| == |y - qy| != 0 is the opposite corner.
- The other two corners are (qx, y) and (x, qy); multiply their counts, since duplicates multiply.

Brute Force

- Try all triples of stored points: O(n^3) per query.

Optimal Approach

- Counts: Map<point, count>; also keep a list of distinct points.
- Count(q): for each distinct diagonal point p: ans += cnt(p) cnt(qx, py) cnt(px, qy).

Worked Example

```
add [3,10] [11,2] [3,2]
count [11,10]: diagonal [3,2] -> cnt[3,2] * cnt[11,2] * cnt[3,10] = 1*1*1 = 1
count [14,8] : no diagonal -> 0
add [11,2] again
count [11,10]: 1 * 2 * 1 = 2
```

Java Solution

```java
public class DetectSquares {
    private final Map<Long, Integer> cnt = new HashMap<>();
    private final List<int[]> distinct = new ArrayList<>();

    private static long key(int x, int y) { return ((long) x << 32) | (y & 0xffffffffL); }

    public void add(int[] p) {
        if (cnt.merge(key(p[0], p[1]), 1, Integer::sum) == 1) distinct.add(p);
    }

    public int count(int[] q) {
        int qx = q[0], qy = q[1], total = 0;
        for (int[] p : distinct) {
            int x = p[0], y = p[1];
            if (Math.abs(x - qx) != Math.abs(y - qy) || x == qx) continue;
            total += cnt.get(key(x, y)) * cnt.getOrDefault(key(qx, y), 0) * cnt.getOrDefault(key(x, qy), 0);
        }
        return total;
    }
}
```

Complexity

- Add O(1), count O(distinct points). Space O(distinct points).

Pitfalls

- Encode points as a single long key to avoid creating many objects.
- Skip zero-area "squares" (same x or same y).